

ASSIGNMENT No 1 — SUMMARY (for hand-copying)

Course: Operating Systems (CE80363)

Lecturer: Placide RWEGO MUCYO

Q1. Computer System Architecture

- **Architecture** = how CPU, memory, I/O devices and buses are organised and work together to execute programs.
 - Modern computers follow the **stored-program (Von Neumann)** concept: program and data are both kept in main memory.
 - **Components:**
 - **CPU** → ALU (calculations), Control Unit (fetches & directs), Registers (PC = next instruction).
 - **Main memory (RAM/ROM)** → holds instructions/data in use (RAM is volatile).
 - **Input / Output devices** → keyboard, monitor, printer, etc.
 - **Secondary storage** → disk, SSD, USB (permanent storage).
 - **System bus** → data bus, address bus, control bus (link all components).
 - **Working:** CPU repeats **Fetch → Decode → Execute** cycle; faster devices are matched by cache, buffers, interrupts, DMA.
 - **Layers:** User → Application → **Operating System** → Hardware.
 - The OS hides hardware complexity and gives a convenient, reliable, secure environment.
 - **Multiprogramming** keeps CPU busy; **timesharing** gives each user quick response (< 1 s).
-

Q2. Operating System Structure

- **Structure** = how OS components are organised and communicate.
- Good structure → efficient, reliable, secure, maintainable.
- **Types:**
- **Simple** - little separation between components; e.g. MS-DOS.
- **Monolithic** - all services in one large kernel (process, memory, file, drivers, network, security); fast but a fault can crash it; e.g. UNIX, Linux.

- **Layered** – divided into layers; each layer serves the one above and uses the one below; easy to design/test, some overhead.
 - **Microkernel** – only essentials in kernel (process, IPC, basic memory, low-level hardware); file systems & drivers run in user space; reliable and safe but slower (message passing).
 - **Modular** – core kernel + loadable modules (drivers, file system); easy to extend; e.g. Linux.
 - **Hybrid** – mix of monolithic + microkernel for performance and modularity; e.g. Windows, Apple XNU.
 - **Virtual machine** – hypervisor runs several virtual computers on one physical machine; full isolation (VMware, VirtualBox).
 - **Trade-offs:** performance, security, reliability, flexibility, maintainability — no structure is perfect.
-

Q3. Major Functions of Operating System

1. **Process Management** – create, schedule, execute, synchronise, terminate processes; states (new, ready, running, waiting, terminated); PCB; context switch; schedulers (long/medium/short term); algorithms FCFS, SJF, Priority, Round Robin; threads; IPC (shared memory, message passing); synchronisation (semaphore, mutex, monitor).
2. **Memory Management** – tracks RAM, allocates/de-allocates memory, address translation, protection, virtual memory, swapping.
3. **File-System Management** – create/delete/read/write files & directories, directory structure, attributes, file protection.
4. **Device (I/O) Management** – device drivers, allocate devices, I/O queues, buffering, caching, spooling, interrupts, device independence.
5. **Security & Protection** – authentication, access control, permissions, protects users and data.
6. **User Interface** – GUI (windows, icons, menus) and CLI (commands/shell).
7. **Communication** – IPC and networking (mail, file transfer, remote login).
8. **Others** – error detection & handling, accounting, job management, command interpretation.

Summary line: The OS manages hardware and software resources, provides services to applications, and acts as the interface between user, applications and hardware.
